

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\integration\IterativeLegendreGaussIntegrator.java

- Specification

module IterativeLegendreGaussIntegrator

type

numberOfPoints: int

relativeAccuracy: real

absoluteAccuracy: real

minimalIterationCount: int

maximalIterationCount: int

min: real

max: real

maxEval: int // general exception set aligned to legacy

ex: set of { None, NullArgumentException, NotStrictlyPositiveException, NumberIsTooSmallException, NumberIsTooLargeException, NotANumberException, NotFiniteNumberException, TooManyEvaluationsException, MaxCountExceededException }

MAX_LEGENDRE_POINTS: int

process integrate(f, min, max, maxEval, numberOfPoints, relativeAccuracy, absoluteAccuracy, minimalIterationCount, maximalIterationCount: real) result: real, ex: set of { ... }

Functional Scenario Form

f = 0 && ex = NullArgumentException ||

maxEval <= 0 && ex = NotStrictlyPositiveException ||

relativeAccuracy <= 0 && ex = NotStrictlyPositiveException ||

absoluteAccuracy <= 0 && ex = NotStrictlyPositiveException ||

minimalIterationCount <= 0 && ex = NotStrictlyPositiveException ||

minimalIterationCount >= maximalIterationCount && ex = NumberIsTooSmallException ||

numberOfPoints < 2 && ex = NumberIsTooSmallException ||

numberOfPoints > MAX_LEGENDRE_POINTS && ex = NumberIsTooLargeException ||

interval = 1 && ex = NotANumberException ||

isInfinite=1 && ex = NotFiniteNumberException ||

min >= max && ex = NumberIsTooLargeException ||

evaluations_used > maxEval && ex = TooManyEvaluationsException ||

iterations_used >= maximalIterationCount && ex = MaxCountExceededException ||

evaluations_used <= maxEval && ex = None

end_process

end_module

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.integration;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
import org.apache.commons.math4.legacy.analysis.integration.gauss.Gauss
Integrator;
import org.apache.commons.math4.legacy.analysis.integration.gauss.Gauss
IntegratorFactory;
import org.apache.commons.math4.legacy.exception.NotStrictlyPositiveExc
eption;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsExce
ption;
import org.apache.commons.math4.legacy.exception.util.LocalizedFormats;
import org.apache.commons.math4.core.jdkmath.JdkMath;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.NotStrictlyPositiveExc
eption;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallExcept
ion;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.legacy.exception.NotANumberException;
```

```

import org.apache.commons.math4.legacy.exception.NotFiniteNumberException;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsException;
import org.apache.commons.math4.legacy.exception.*;
/**
 * This algorithm divides the integration interval into equally-sized
 * sub-interval and on each of them performs a
 * Legendre-Gauss quadrature. Because of its non-adaptive nature,
 * this algorithm can converge to a wrong value for the integral (for example,
 * if the function is significantly different from zero toward the ends of the
 * integration interval). In particular, a change of variables aimed at
 * estimating integrals over infinite intervals as proposed here should be avoided when using this class.
 *
 * @since 3.1
 */
public class IterativeLegendreGaussIntegrator extends BaseAbstractUnivariateIntegrator {
    /** Factory that computes the points and weights. */
    private static final GaussIntegratorFactory FACTORY = new GaussIntegratorFactory();
    /** Number of integration points (per interval). */
    private final int numberOfPoints;
    /**
     * Builds an integrator with given accuracies and iterations counts.
     *
     * @param n Number of integration points.
     * @param relativeAccuracy Relative accuracy of the result.
     * @param absoluteAccuracy Absolute accuracy of the result.
     * @param minimalIterationCount Minimum number of iterations.
     * @param maximalIterationCount Maximum number of iterations.
     * @throws NotStrictlyPositiveException if
     *
     * minimal

```

*
number
*
of
*
iterations
*
or
*
number
*
of
*
points
*
are
*
not
*
strictly
*
positive.
* @throws org.apache.commons.math4.legacy.exception.NumberIsTooSmallException if
*
the
*
maximal
*
number
*
of
*
iterations
*
is
*
smaller
*
than
*
or
*
equal

```

*
to
*
the
*
minimal
*
number
*
of
*
iterations.
*/
public IterativeLegendreGaussIntegrator(final int n, final double relativeAccuracy, final double absoluteAccuracy,
    final int minimalIterationCount, final int maximalIterationCount) {
    super(relativeAccuracy, absoluteAccuracy, minimalIterationCount, maximalIterationCount);
    if (n <= 0) {
        throw new NotStrictlyPositiveException(LocalizedFormats.NUMBER_OF_POINTS, n);
    }
    numberOfPoints = n;
}
/**
 * Builds an integrator with given accuracies.
 *
 * @param n          Number of integration points.
 * @param relativeAccuracy Relative accuracy of the result.
 * @param absoluteAccuracy Absolute accuracy of the result.
 * @throws NotStrictlyPositiveException if {@code n < 1}.
 */
public IterativeLegendreGaussIntegrator(final int n, final double relativeAccuracy, final double absoluteAccuracy) {
    this(n, relativeAccuracy, absoluteAccuracy, DEFAULT_MIN_ITERATIONS_COUNT, DEFAULT_MAX_ITERATIONS_COUNT);
}
/**
 * Builds an integrator with given iteration counts.
 *
 * @param n          Number of integration points.
 * @param minimalIterationCount Minimum number of iterations.
 * @param maximalIterationCount Maximum number of iterations.

```

```
* @throws NotStrictlyPositiveException
    if
*
minimal
*
number
*
of
*
iterations
*
is
*
not
*
strictly
*
positive.
* @throws org.apache.commons.math4.legacy.exception.NumberIsTooSmallE
xception if
*
the
*
maximal
*
number
*
of
*
iterations
*
is
*
smaller
*
than
*
or
*
equal
*
to
*
the
```

```

    *
    minimal
    *
    number
    *
    of
    *
    iterations.
    * @throws NotStrictlyPositiveException
        if
    *
    {@code n < 1}.
    */
    public IterativeLegendreGaussIntegrator(final int n, final int minimal
IterationCount,
        final int maximalIterationCount) {
        this(n, DEFAULT_RELATIVE_ACCURACY, DEFAULT_ABSOLUTE_ACCURACY, minimal
IterationCount, maximalIterationCount);
    }
    /** {@inheritDoc} */
    @Override
    protected double doIntegrate() {
        // Compute first estimate with a single step.
        double oldt = stage(1);
        int n = 2;
        while (true) {
            // Improve integral with a larger number of steps.
            final double t = stage(n);
            // Estimate the error.
            final double delta = JdkMath.abs(t - oldt);
            final double limit = JdkMath.max(getAbsoluteAccuracy(),
                getRelativeAccuracy() * (JdkMath.abs(oldt) + JdkMath.abs(t)) * 0.5
);
            // check convergence
            if (iterations.getCount() + 1 >= getMinimalIterationCount() && delta
<= limit) {
                return t;
            }
            // Prepare next iteration.
            final double ratio = JdkMath.min(4, JdkMath.pow(delta / limit, 0.5 /
numberOfPoints));
            n = JdkMath.max((int) (ratio * n), n + 1);
            oldt = t;
            iterations.increment();

```

```

    }
}
/**
 * Compute the n-th stage integral.
 *
 * @param n Number of steps.
 * @return the value of n-th stage integral.
 * @throws TooManyEvaluationsException if the maximum number of evalua
tions is
 *
 * exceeded.
 */
private double stage(final int n) throws TooManyEvaluationsException {
    // Function to be integrated is stored in the base class.
    final UnivariateFunction f = new UnivariateFunction() {
        /** {@inheritDoc} */
        @Override
        public double value(double x) {
            return computeObjectiveValue(x);
        }
    };
    final double min = getMin();
    final double max = getMax();
    final double step = (max - min) / n;
    double sum = 0;
    for (int i = 0; i < n; i++) {
        // Integrate over each sub-interval [a, b].
        final double a = min + i * step;
        final double b = a + step;
        final GaussIntegrator g = FACTORY.legendreHighPrecision(numberOfPoin
ts, a, b);
        sum += g.integrate(f);
    }
    return sum;
}
}
class ILGVT {
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ====");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expe
cted) {
        System.out.println("---- " + name + " ----");
        try {

```

```

        r.run();
        System.out.printf("[FAIL] %s should throw %s%n", name, expected.getSimpleName());
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
        System.out.printf("[%s] %s threw expected%n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
    }
}

// ----- sample functions -----
static final UnivariateFunction F_SIN = Math::sin; //  $\int_0^\pi \sin = 2$ 
static final UnivariateFunction F_LIN = x -> x - 2.0; //  $\int_a^b (x-2) dx = (b^2-a^2)/2 - 2(b-a)$ 
static final UnivariateFunction F_CONST = x -> 5.0; //  $\int_a^b 5 dx = 5(b-a)$ 
static final UnivariateFunction F_ZERO = x -> 0.0;
public void Vtest() {
    // Choose a "known good" configuration (nodes, accuracies, iteration band).
    final int NP_OK = 32; // typical safe Gauss-Legendre nodes
    final int MIN_ITER = 3; // strictly > 0
    final int MAX_ITER = 8; // strictly > MIN_ITER
    final double REL = 1e-10; // > 0
    final double ABS = 1e-10; // > 0
    IterativeLegendreGaussIntegrator G = new IterativeLegendreGaussIntegrator(NP_OK, REL, ABS, MIN_ITER, MAX_ITER);
    // VT1: f == null -> NullArgumentException
    expectThrow("VT1 f=null", () -> G.integrate(1000, null, 0.0, 1.0), NullArgumentException.class);
    // VT2: maxEval <= 0 -> NotStrictlyPositiveException
    expectThrow("VT2 maxEval=0", () -> G.integrate(0, F_SIN, 0.0, 1.0), NotStrictlyPositiveException.class);
    expectThrow("VT2b maxEval<0", () -> G.integrate(-5, F_SIN, 0.0, 1.0), NotStrictlyPositiveException.class);
    // VT3: relativeAccuracy <= 0 (ctor) -> NotStrictlyPositiveException
    expectThrow("VT3 relAcc<=0", () -> new IterativeLegendreGaussIntegrator(NP_OK, 0.0, 1e-8, MIN_ITER, MAX_ITER), NotStrictlyPositiveException.class);
    // VT4: absoluteAccuracy <= 0 (ctor) -> NotStrictlyPositiveException
    expectThrow("VT4 absAcc<=0", () -> new IterativeLegendreGaussIntegrator(NP_OK, 1e-8, 0.0, MIN_ITER, MAX_ITER), NotStrictlyPositiveException.class);
}

```

```

// VT5: minimalIterationCount <= 0 (ctor) -> NotStrictlyPositiveException
expectThrow("VT5 minIter<=0", () -> new IterativeLegendreGaussIntegrator(NP_OK, REL, ABS, 0, MAX_ITER),
    NotStrictlyPositiveException.class);
// VT6: minimalIterationCount >= maximalIterationCount (strict) ->
// NumberIsTooSmallException
expectThrow("VT6 minIter>=maxIter (strict)", () -> new IterativeLegendreGaussIntegrator(NP_OK, REL, ABS, 5, 5),
    NumberIsTooSmallException.class);
// VT7: numberOfPoints < 2 -> NumberIsTooSmallException
expectThrow("VT7 numberOfPoints<2", () -> new IterativeLegendreGaussIntegrator(1, REL, ABS, MIN_ITER, MAX_ITER),
    NumberIsTooSmallException.class);
// VT8: numberOfPoints extremely large -> expect NumberIsTooLargeException OR
// note no enforcement
try {
    new IterativeLegendreGaussIntegrator(10_000, REL, ABS, MIN_ITER, MAX_ITER);
    System.out.println("VT8: constructed with nPoints=10000 (no upper bound enforcement in this build).");
} catch (NumberIsTooLargeException e) {
    System.out.println("VT8: threw NumberIsTooLargeException as expected for large nPoints.");
} catch (Throwable t) {
    System.out.println("VT8: unexpected throwable: " + t);
}
// VT9: NaN endpoints -> NotANumberException
expectThrow("VT9 NaN min", () -> G.integrate(1000, F_LIN, Double.NaN, 1.0), NotANumberException.class);
expectThrow("VT9b NaN max", () -> G.integrate(1000, F_LIN, 0.0, Double.NaN), NotANumberException.class);
// VT10: Infinite endpoints -> NotFiniteNumberException
expectThrow("VT10 -Infinity at min", () -> G.integrate(1000, F_SIN, Double.NEGATIVE_INFINITY, 1.0),
    NotFiniteNumberException.class);
expectThrow("VT10b +Infinity at max", () -> G.integrate(1000, F_SIN, 0.0, Double.POSITIVE_INFINITY),
    NotFiniteNumberException.class);
// VT11: min >= max (beta1 enforces strict min<max) -> NumberIsTooLargeException

```

```

    expectThrow("VT11 min==max", () -> G.integrate(1000, F_CONST, 3.14, 3
.14), NumberIsTooLargeException.class);
    expectThrow("VT11b min>max", () -> G.integrate(1000, F_CONST, 1.0, -
1.0), NumberIsTooLargeException.class);
    // VT12: budget exhausted (tiny maxEval on nontrivial interval) ->
    // TooManyEvaluationsException
    expectThrow("VT12 budget exhausted (maxEval=1)", () -> G.integrate(1,
F_SIN, 0.0, 10.0),
        TooManyEvaluationsException.class);
    // ---- Success paths ----
    // VT13: success on ZERO function, small interval, budget big enough
for
    // multi-stage GL
    headline("VT13 ZERO small interval within budget");
    int budget13 = NP_OK * (2 * MAX_ITER + 10); // e.g., 32 * (2*8 + 10)
= 832
    double r13 = G.integrate(budget13, F_ZERO, -1e-6, 1e-6);
    System.out.printf("VT13 result=%.17g (budget=%d, nPoints=%d, minIter=
%d, maxIter=%d)%n", r13, budget13, NP_OK,
        MIN_ITER, MAX_ITER);
    // VT14: success on CONST small interval, budget scaled similarly
    headline("VT14 CONST small interval");
    double a14 = 2.0, b14 = 2.000000001;
    int budget14 = NP_OK * (2 * MAX_ITER + 10);
    double r14 = G.integrate(budget14, F_CONST, a14, b14);
    System.out.printf("VT14 result=%.17g (expected=%g, budget=%d)%n", r14
, 5.0 * (b14 - a14), budget14);
    // VT15: success on SIN over [0,  $\pi$ ]  $\approx 2$ 
    headline("VT15 SIN [0, $\pi$ ]");
    double r15 = G.integrate(20000, F_SIN, 0.0, Math.PI);
    System.out.printf("VT15 result=%.17g |err|=%g%n", r15, Math.abs(r15 -
2.0));
    // VT16: success on LIN over [0,4] = 0
    headline("VT16 LIN [0,4]");
    double r16 = G.integrate(20000, F_LIN, 0.0, 4.0);
    System.out.printf("VT16 result=%.17g |err|=%g%n", r16, Math.abs(r16 -
0.0));
    // VT17: edge – very tight accuracies but feasible iteration band
    headline("VT17 tight accuracies");
    IterativeLegendreGaussIntegrator Gt = new IterativeLegendreGaussInteg
rator(NP_OK, 1e-12, 1e-12, 3, 12);
    double r17 = Gt.integrate(500000, F_SIN, 0.0, Math.PI);
    System.out.printf("VT17 result=%.17g |err|=%g%n", r17, Math.abs(r17 -
2.0));

```

```

// VT18: smallest allowed nodes (nPoints=2) - success on simple function
headline("VT18 nPoints=2 basic success");
IterativeLegendreGaussIntegrator G2 = new IterativeLegendreGaussIntegrator(2, REL, ABS, MIN_ITER, MAX_ITER);
double r18 = G2.integrate(1000, F_CONST, -1.0, 1.0);
System.out.printf("VT18 result=%.17g (expected ≈ 10)%n", r18);
// VT19: near-degenerate but valid interval (ensure spacing > nPoints * ulp(a))
// Reason: GaussIntegrator requires strictly increasing mapped nodes; if (b-a)
// is ~ 0,
// multiple mapped nodes collapse to the same double value ->
// NonMonotonicSequenceException.
headline("VT19 near-degenerate interval (robust)");
double a19 = 3.0;
// ensure the interval width is safely larger than the accumulated rounding:
// use max of a few ULPs and a small absolute epsilon.
double eps19 = Math.max(1e-9, NP_OK * Math.ulp(a19)); // NP_OK = numberOfPoints
double b19 = a19 + eps19;
int budget19 = NP_OK * (2 * MAX_ITER + 10); // keep generous budget like VT13/VT14
double r19 = G.integrate(budget19, F_CONST, a19, b19);
System.out.printf("VT19 result=%.17g (expected≈%g, eps=%g)%n", r19, 5.0 * eps19, eps19);
// VT20: sanity on wide finite interval with large budget (may be slow; ok for harness)
headline("VT20 wide finite SIN sanity [-100,100]");
double r20 = G.integrate(2_000_000, F_SIN, -100.0, 100.0);
System.out.printf("VT20 result=%.17g%n", r20);
System.out.println("\n[Done] IterativeLegendreGaussIntegrator V-method cases executed.");
}
}
//FT
class ILGFT {
// ----- helpers -----
private static void headline(String name) {
System.out.println("\n==== " + name + " ====");
}
}

```

```

private static void expectThrow(String name, Runnable r, Class<?> expected) {
    System.out.println("---- " + name + " ----");
    try {
        r.run();
        System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
        System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
    }
}

// ----- sample functions -----
static final UnivariateFunction F_SIN = Math::sin; //  $\int_0^\pi \sin = 2$ 
static final UnivariateFunction F_LIN = x -> x - 2.0; //  $\int_a^b (x-2) dx = (b^2-a^2)/2 - 2(b-a)$ 
static final UnivariateFunction F_CONST = x -> 5.0; //  $\int_a^b 5 dx = 5(b-a)$ 
static final UnivariateFunction F_ZERO = x -> 0.0;
public void Ftest() {
    // Choose a "known good" configuration (nodes, accuracies, iteration band).
    final int NP_OK = 32; // typical safe Gauss-Legendre nodes
    final int MIN_ITER = 3; // strictly > 0
    final int MAX_ITER = 8; // strictly > MIN_ITER
    final double REL = 1e-10; // > 0
    final double ABS = 1e-10; // > 0
    IterativeLegendreGaussIntegrator G = new IterativeLegendreGaussIntegrator(NP_OK, REL, ABS, MIN_ITER, MAX_ITER);
    // ----- Fuzz pack: FZ01 - FZ20 -----
    int bud = NP_OK * (2 * MAX_ITER + 20);
    // FZ01:
    headline("FZ01 SIN large finite interval");
    double fz01 = G.integrate(bud, F_SIN, -1.23456789012345E2, 3.2109876543210E2);
    System.out.printf("FZ01 result=%.17g\n", fz01);
    // FZ02:
    expectThrow("FZ02 degenerate interval (min==max)",
        () -> G.integrate(bud, F_CONST, 9.876543210123456E1, 9.876543210123456E1),
        NumberIsTooLargeException.class);
    // FZ03:

```

```

headline("FZ03 LIN medium interval");
double fz03 = G.integrate(bud, F_LIN, -7.5, 13.75);
System.out.printf("FZ03 result=%.17g%n", fz03);
// FZ04:
expectThrow("FZ04 reversed interval", () -> G.integrate(bud, F_SIN, 5
.005341669616643E3, -1.12091454291415E3),
    NumberIsTooLargeException.class);
// FZ05:
headline("FZ05 ZERO tiny symmetric interval");
double fz05 = G.integrate(bud, F_ZERO, -1e-8, 1e-8);
System.out.printf("FZ05 result=%.17g%n", fz05);
// FZ06:
headline("FZ06 CONST near-degenerate safe");
double a06 = 3.141592653589793;
double eps06 = Math.max(1e-9, NP_OK * Math.ulp(a06));
double fz06 = G.integrate(bud, F_CONST, a06, a06 + eps06);
System.out.printf("FZ06 result=%.17g (expected=%g, eps=%g)%n", fz06,
5.0 * eps06, eps06);
// FZ07:
expectThrow("FZ07 NaN min", () -> G.integrate(bud, F_LIN, Double.NaN,
1.0), NotANumberException.class);
// FZ08:
expectThrow("FZ08 +Infinity at max", () -> G.integrate(bud, F_SIN, -
10.0, Double.POSITIVE_INFINITY),
    NotFiniteNumberException.class);
// FZ09:
headline("FZ09 SIN on  $[-\pi, \pi]$ ");
double fz09 = G.integrate(bud, F_SIN, -Math.PI, Math.PI);
System.out.printf("FZ09 result=%.17g%n", fz09);
// FZ10:
headline("FZ10 CONST wide finite");
double fz10 = G.integrate(bud, F_CONST, -1.2345E4, 1.9999E4);
System.out.printf("FZ10 result=%.17g (expected=%g)%n", fz10, 5.0 * (1
.9999E4 + 1.2345E4));
// FZ11:
expectThrow("FZ11 tiny budget -> TooManyEvaluations", () -> G.integra
te(1, F_SIN, 0.0, 100.0),
    TooManyEvaluationsException.class);
// FZ12:
headline("FZ12 LIN small asymmetric");
double fz12 = G.integrate(bud, F_LIN, -2.5e-3, 7.0e-3);
System.out.printf("FZ12 result=%.17g%n", fz12);
// FZ13:
headline("FZ13 SIN on  $[0, 10\pi]$ ");

```

```

double fz13 = G.integrate(bud, F_SIN, 0.0, 10.0 * Math.PI);
System.out.printf("FZ13 result=%.17g%n", fz13);
// FZ14:
expectThrow("FZ14 -
Infinity at min", () -> G.integrate(bud, F_CONST, Double.NEGATIVE_INFINITY, -1.0),
    NotFiniteNumberException.class);
// FZ15:
headline("FZ15 ZERO tiny shifted");
double fz15 = G.integrate(bud, F_ZERO, 2.718281828459045e-
9, 3.718281828459045e-9);
System.out.printf("FZ15 result=%.17g%n", fz15);
// FZ16:
headline("FZ16 CONST near-degenerate negative");
double a16 = -7.0;
double eps16 = Math.max(1e-9, NP_OK * Math.ulp(a16));
double fz16 = G.integrate(bud, F_CONST, a16, a16 + eps16);
System.out.printf("FZ16 result=%.17g (expected=%g, eps=%g)%n", fz16,
5.0 * eps16, eps16);
// FZ17:
headline("FZ17 SIN randomish window");
double fz17 = G.integrate(bud, F_SIN, 12.3456789012345, 23.4567890123
456);
System.out.printf("FZ17 result=%.17g%n", fz17);
// FZ18:
expectThrow("FZ18 maxEval<=0", () -> G.integrate(0, F_LIN, -
1.0, 1.0), NotStrictlyPositiveException.class);
// FZ19:
headline("FZ19 LIN wide finite");
double fz19 = G.integrate(bud, F_LIN, -
5.43210987654321E2, 7.65432109876543E2);
System.out.printf("FZ19 result=%.17g%n", fz19);
// FZ20:
headline("FZ20 SIN very wide finite");
double fz20 = G.integrate(bud, F_SIN, -1.0e3, 1.0e3);
System.out.printf("FZ20 result=%.17g%n", fz20);
System.out.println("\n[Done] IterativeLegendreGaussIntegrator Fuzzing
cases executed.");
}
}
// Z3
class ILGZ3 {
private static void headline(String name) {
System.out.println("\n==== " + name + " ====");

```

```

    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    // ---- functions ----
    static final UnivariateFunction F_ZERO = x -> 0.0;
    static final UnivariateFunction F_CONST = x -> 5.0;
    static final UnivariateFunction F_SIN = Math::sin;
    static final UnivariateFunction F_LIN = x -> x - 2.0;
    public void Z3test() {
        final int NP_SAFE = 2;
        final double REL_SAFE = 1.0 / 1000.0;
        final double ABS_SAFE = 1.0 / 1000.0;
        final int MIN_SAFE = 1;
        final int MAX_SAFE = 2;
        // Z301 -----
        -----
        /*
         * 01) f = null → ex = NullPointerException numberOfPoints=0, rel=1/1000,
         * abs=1/1000, minIter=0, maxIter=0, min=0, max=1, maxEval=0,
         * MAX_LEGENDRE_POINTS=8 f_is_null=True ... ex = NullPointerException (code=1)
         */
        IterativeLegendreGaussIntegrator G301 = new IterativeLegendreGaussIntegrator(NP_SAFE, REL_SAFE, ABS_SAFE, MIN_SAFE, MAX_SAFE);
        expectThrow("Z301 f=null", () -> G301.integrate(10, null, 0.0, 1.0), NullPointerException.class);
        // Z302 -----
        -----
        /*

```

```

    * 02) maxEval <= 0 → ex = NotStrictlyPositiveException numberOfPoints=2,
    * rel=1/1000, abs=1/1000, minIter=1, maxIter=2, min=0, max=1, maxEval=0
    */
    IterativeLegendreGaussIntegrator G302 = new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 1, 2);
    expectThrow("Z302 maxEval<=0", () -> G302.integrate(0, F_ZERO, 0.0, 1.0), NotStrictlyPositiveException.class);
    // Z303 -----
    -----
    /*
    * 03) relativeAccuracy <= 0 → ex = NotStrictlyPositiveException [WARN] UNSAT –
    * could not find a model for this scenario.
    */
    expectThrow("Z303 relAcc<=0 (ctor)", () -> new IterativeLegendreGaussIntegrator(2, 0.0, ABS_SAFE, 1, 2),
        NotStrictlyPositiveException.class);
    // Z304 -----
    -----
    /*
    * 04) absoluteAccuracy <= 0 → ex = NotStrictlyPositiveException [WARN] UNSAT –
    * could not find a model for this scenario.
    */
    expectThrow("Z304 absAcc<=0 (ctor)", () -> new IterativeLegendreGaussIntegrator(2, REL_SAFE, 0.0, 1, 2),
        NotStrictlyPositiveException.class);
    // Z305 -----
    -----
    /*
    * 05) minimalIterationCount <= 0 → ex = NotStrictlyPositiveException
    * numberOfPoints = 2 ... minIter = 0, maxIter = 1
    */
    expectThrow("Z305 minIter<=0 (ctor)", () -> new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 0, 1),
        NotStrictlyPositiveException.class);
    // Z306 -----
    -----
    /*
    * 06) minimalIterationCount >= maximalIterationCount → ex =
    * NumberIsTooSmallException minIter=1, maxIter=0
    */

```

```

expectThrow("Z306 minIter>=maxIter (ctor)",
    () -> new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 1
, 0),
    NumberIsTooSmallException.class);
// Z307 -----
-----
/*
 * 07) numberOfPoints < 2 → ex = NumberIsTooSmallException numberOfPo
ints=0
 */
expectThrow("Z307 numberOfPoints<2 (ctor)",
    () -> new IterativeLegendreGaussIntegrator(0, REL_SAFE, ABS_SAFE, 1
, 2),
    NumberIsTooSmallException.class);
// Z308 -----
-----
/*
 * 08) numberOfPoints > MAX_LEGENDRE_POINTS → ex = NumberIsTooLargeEx
ception
 * numberOfPoints=9, MAX_LEGENDRE_POINTS=8
 */
try {
    new IterativeLegendreGaussIntegrator(9, REL_SAFE, ABS_SAFE, 1, 2);
    System.out.println("Z308: nPoints=9 constructed (this build does not
enforce MAX=8).");
} catch (NumberIsTooLargeException e) {
    System.out.println("Z308: threw NumberIsTooLargeException as expecte
d for nPoints>MAX.");
}
// Z309 -----
-----
/*
 * 09) isNaN(min) → ex = NotANumberException
 */
IterativeLegendreGaussIntegrator G309 = new IterativeLegendreGaussInt
egrator(2, REL_SAFE, ABS_SAFE, 1, 2);
expectThrow("Z309 NaN(min)", () -> G309.integrate(10, F_LIN, Double.N
aN, 1.0), NotANumberException.class);
// Z310 -----
-----
/*
 * 10) isNaN(max) → ex = NotANumberException
 */

```

```

    IterativeLegendreGaussIntegrator G310 = new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 1, 2);
    expectThrow("Z310 NaN(max)", () -> G310.integrate(10, F_LIN, 0.0, Double.NaN), NotANumberException.class);
    // Z311 -----
    -----
    /*
    * 11) isInfinite(min) → ex = NotFiniteNumberException
    */
    IterativeLegendreGaussIntegrator G311 = new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 1, 2);
    expectThrow("Z311 -Infinity(min)", () -> G311.integrate(10, F_SIN, Double.NEGATIVE_INFINITY, 1.0),
        NotFiniteNumberException.class);
    // Z312 -----
    -----
    /*
    * 12) isInfinite(max) → ex = NotFiniteNumberException
    */
    IterativeLegendreGaussIntegrator G312 = new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 1, 2);
    expectThrow("Z312 +Infinity(max)", () -> G312.integrate(10, F_SIN, 0.0, Double.POSITIVE_INFINITY),
        NotFiniteNumberException.class);
    // Z313 -----
    -----
    /*
    * 13)  $\min \geq \max \rightarrow \text{ex} = \text{NumberIsTooLargeException}$  [WARN] UNSAT – could not find
    * a model for this scenario. — 实际实现:  $\min == \max$  /  $\min > \max$  均由区间校验抛
    * NumberIsTooLargeException
    */
    IterativeLegendreGaussIntegrator G313 = new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 1, 2);
    expectThrow("Z313  $\min \geq \max$  ( $\min == \max$ )", () -> G313.integrate(10, F_CONST, 0.0, 0.0),
        NumberIsTooLargeException.class);
    // Z314 -----
    -----
    /*
    * 14) valid & evaluations_used > maxEval → ex = TooManyEvaluationsException

```

```

    * maxEval=10, evaluations_used=11
    */
    IterativeLegendreGaussIntegrator G314 = new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 1, 2);
    expectThrow("Z314 budget exhausted (maxEval=1)", () -> G314.integrate(1, F_SIN, 0.0, 1.0), TooManyEvaluationsException.class);
    // Z315 -----
    -----
    /*
    * 15) valid & iterations_used ≥ maximalIterationCount & !converged → ex =
    * MaxCountExceededException
    */
    try {
        IterativeLegendreGaussIntegrator G315 = new IterativeLegendreGaussIntegrator(2, 1e-30, 1e-30, 1, 2); // 极紧精度, 小上限
        expectThrow("Z315 hit max iterations (tight accuracy, small band)", () -> G315.integrate(10_000, F_SIN, 0.0, 100.0), MaxCountExceededException.class);
    } catch (Throwable t) {
        System.out.println("Z315 note: " + t.getClass().getSimpleName() + " - " + t.getMessage());
    }
    // Z316 -----
    -----
    /*
    * 16) valid & converged_by_interval_shrinkage → ex = None
    */
    headline("Z316 success (interval shrinkage)");
    IterativeLegendreGaussIntegrator G316 = new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 1, 2);
    double z316 = G316.integrate(100, F_CONST, 0.0, 1e-6);
    System.out.printf("Z316 result=%.17g%n", z316);
    // Z317 -----
    -----
    /*
    * 17) valid & converged_by_value_accuracy → ex = None
    */
    headline("Z317 success (value accuracy)");
    IterativeLegendreGaussIntegrator G317 = new IterativeLegendreGaussIntegrator(2, 1e-3, 1e-12, 1, 2); // 极小 absAcc
    double z317 = G317.integrate(100, F_SIN, 0.0, 1e-3);
    System.out.printf("Z317 result=%.17g%n", z317);

```

```

// Z318 -----
-----
/*
 * 18) valid & numberOfPoints = 2 (boundary) → ex = None
 */
headline("Z318 nPoints=2 boundary success");
IterativeLegendreGaussIntegrator G318 = new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 1, 2);
double z318 = G318.integrate(100, F_ZERO, 0.0, 1.0);
System.out.printf("Z318 result=%.17g%n", z318);
// Z319 -----
-----
/*
 * 19) valid & numberOfPoints = MAX (boundary) → ex = None
 * MAX_LEGENDRE_POINTS=16
 */
headline("Z319 nPoints=16 boundary (build-dependent)");
try {
    IterativeLegendreGaussIntegrator G319 = new IterativeLegendreGaussIntegrator(16, REL_SAFE, ABS_SAFE, 1, 2);
    double z319 = G319.integrate(200, F_ZERO, 0.0, 1.0);
    System.out.printf("Z319 result=%.17g%n", z319);
} catch (NumberIsTooLargeException e) {
    System.out.println("Z319: build enforces MAX<nPoints=16>; threw as expected.");
}
// Z320 -----
-----
/*
 * 20) valid & minimalIterationCount = 1 (quick converge) → ex = None
 */
headline("Z320 minIter=1 quick converge");
IterativeLegendreGaussIntegrator G320 = new IterativeLegendreGaussIntegrator(2, REL_SAFE, ABS_SAFE, 1, 2);
double z320 = G320.integrate(100, F_CONST, 0.0, 1e-4);
System.out.printf("Z320 result=%.17g%n", z320);
System.out.println("\n[Done] Z301-Z320 executed.");
}
}
class ILGtest {
    public static void main(String[] args) {
        // VT
        ILGVT t1 = new ILGVT();
        t1.Vtest();
    }
}

```

```

// FT
ILGFT t2 = new ILGFT();
// t2.Ftest();
// Z3
ILGZ3 t3 = new ILGZ3();
// t3.Z3test();
}
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
IterativeLegendreGaussIntegrator.java	85.7 %	12	2	14
> ILGVT	0.0 %	0	2	2
> ILGFT	100.0 %	2	0	2
> ILGZ3	100.0 %	2	0	2
> IterativeLegendreGaussIntegrator	100.0 %	8	0	8
> IterativeLegendreGaussIntegrator(in	100.0 %	2	0	2
doIntegrate()	100.0 %	4	0	4
> stage(int)	100.0 %	2	0	2
> ILGtest		0	0	0

3. V-Method testing instruction coverage: 90.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
IterativeLegendreGaussIntegrator.java	57.7 %	1,082	794	1,876
> ILGVT	2.0 %	12	593	605
> ILGZ3	79.1 %	436	115	551
> ILGtest	85.0 %	17	3	20
> ILGFT	87.2 %	458	67	525
> IterativeLegendreGaussIntegrator	90.2 %	148	16	164
> IterativeLegendreGaussIntegrator(in	100.0 %	19	0	19
doIntegrate()	100.0 %	71	0	71
> stage(int)	100.0 %	53	0	53

4. Fuzz testing branch coverage: 87.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
IterativeLegendreGaussIntegrator.java	64.3 %	9	5	14
> ILGFT	0.0 %	0	2	2
> ILGZ3	0.0 %	0	2	2
> IterativeLegendreGaussIntegrator	87.5 %	7	1	8
> IterativeLegendreGaussIntegrator(in	50.0 %	1	1	2
doIntegrate()	100.0 %	4	0	4
> stage(int)	100.0 %	2	0	2
> ILGVT	100.0 %	2	0	2
> ILGtest		0	0	0

5. Fuzz testing instruction coverage: 86%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
IterativeLegendreGaussIntegrator.java	34.6 %	649	1,225	1,874
> ILGVT	2.0 %	12	593	605
> ILGZ3	2.2 %	12	539	551
> ILGtest	83.3 %	15	3	18
> IterativeLegendreGaussIntegrator	86.0 %	141	23	164
> IterativeLegendreGaussIntegrator(in	63.2 %	12	7	19
doIntegrate()	100.0 %	71	0	71
> stage(int)	100.0 %	53	0	53
> ILGFT	87.2 %	458	67	525

6. Z3 branch coverage: 87.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
IterativeLegendreGaussIntegrator.java	64.3 %	9	5	14
> ILGFT	0.0 %	0	2	2
> ILGZ3	0.0 %	0	2	2
IterativeLegendreGaussIntegrator	87.5 %	7	1	8
IterativeLegendreGaussIntegrator(in	50.0 %	1	1	2
doIntegrate()	100.0 %	4	0	4
stage(int)	100.0 %	2	0	2
ILGVT	100.0 %	2	0	2
ILGtest		0	0	0

7. Z3 instruction coverage: 86%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
IterativeLegendreGaussIntegrator.java	37.0 %	693	1,181	1,874
> ILGZ3	2.2 %	12	539	551
> ILGFT	2.3 %	12	513	525
> ILGVT	83.0 %	502	103	605
> ILGtest	83.3 %	15	3	18
IterativeLegendreGaussIntegrator	86.0 %	141	23	164
IterativeLegendreGaussIntegrator(in	63.2 %	12	7	19
doIntegrate()	100.0 %	71	0	71
stage(int)	100.0 %	53	0	53